# Digital VLSI Assignment: 4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Lakshmana Venkatachary M
Roll Number: 2026702012
**GitHub Repository:** https://github.com/mlvenkatac/mult4_2012

October 26, 2026

## 1 Introduction

This report presents the complete RTL-to-GDSII implementation of a 4-bit by 4-bit unsigned array multiplier producing an 8-bit registered product, designed and implemented on the SkyWater 130 nm CMOS process (`sky130_fd_sc_hd` high-density standard cell library).

The design architecture consists of 16 two-input AND gates to generate four 4-bit partial products ($pp_0$ through $pp_3$), followed by three cascaded ripple-carry adder rows containing 12 structural full adders in total. To enable synchronous static timing analysis (STA) and clock tree synthesis (CTS), both the 4-bit inputs ($a[3:0]$ and $b[3:0]$) and the 8-bit product output ($p[7:0]$) are buffered using positive-edge-triggered D-flip-flops with active-low asynchronous reset (`rst_n`). This pipeline structure requires 16 flip-flops in total, yielding a deterministic latency of 2 clock cycles and a steady-state throughput of 1 multiplication result per clock cycle.

The complete open-source ASIC flow is executed inside the `IIC-OSIC-TOOLS` container environment:

- **Part A (RTL Design & Verification):** Structural Verilog implementation (`full_adder.v` and `mult_array.v`) and exhaustive verification across all 256 input vectors using an automated self-checking testbench (`tb_mult.v`).
- **Part B (Logic Synthesis):** Technology mapping of the flattened RTL to SkyWater 130 nm standard cells using Yosys, followed by gate-level functional netlist verification.
- **Part C (Static Timing Analysis):** Pre-layout critical path extraction and maximum operating frequency ($F_{\max}$) evaluation at the typical process corner (`tt_025C_1v80`) using OpenSTA.
- **Part D (Physical Design):** Automated floorplanning, standard-cell placement, clock tree synthesis, routing, parasitic extraction, and physical signoff (DRC and LVS verification) to generate the final GDSII layout (`mult_array.gds`) using LibreLane and OpenROAD.

## 2 Design and Verification (Part A)

### 2.1 RTL Architecture

The 4-bit unsigned array multiplier is partitioned into two Verilog modules:

1. `rtl/full_adder.v`: A structural 1-bit full adder computing sum $= a \oplus b \oplus c_{\text{in}}$ and $c_{\text{out}} = (a \cdot b) + (c_{\text{in}} \cdot (a \oplus b))$ using continuous `assign` statements.

2. `rtl/mult_array.v`: The top-level synchronous multiplier module with inputs `clk`, `rst_n` (active-low asynchronous reset), $a[3:0]$, $b[3:0]$, and registered output $p[7:0]$.

Inside `mult_array.v`, inputs $a[3:0]$ and $b[3:0]$ are captured into two 4-bit internal registers (`a_reg` and `b_reg`) on the rising edge of `clk` (8 flip-flops). Sixteen 2-input AND gates compute the four 4-bit partial products $pp_i[j] = \texttt{a_reg}[j]$ & $\texttt{b_reg}[i]$ for $i, j \in \{0, 1, 2, 3\}$. Row 0 directly outputs $p_{\text{comb}}[0] = pp_0[0]$ and passes $r_0 = \{0, pp_0[3:1]\}$ to Row 1. Rows 1, 2, and 3 are each constructed as a 4-bit ripple-carry adder using 4 instances of `full_adder` per row (12 full adders in total), with the initial carry-in of each row tied to 0 ($c_1[0] = c_2[0] = c_3[0] = 0$). Finally, the 8-bit combinational product $p_{\text{comb}}[7:0]$ is captured into the 8-bit output register $p[7:0]$ on the next rising clock edge (8 flip-flops), resulting in 16 flip-flops total and a 2-clock-cycle pipeline latency.

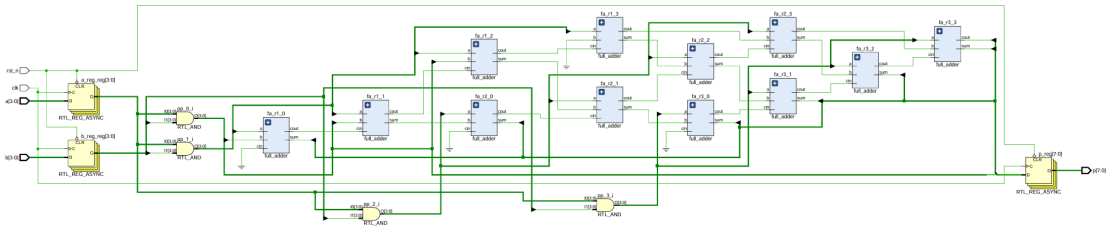


Figure 1: Elaborated RTL block diagram of `mult_array` showing the input registers (`a_reg`, `b_reg`), 16 AND gates (`pp_0` to `pp_3`), 12 structural full adders across Rows 1–3, and the 8-bit output register (`p_reg`).

## 2.2 Hand-Worked Example (Task 5)

Since the last digit of my roll number is 2, the custom test values specified by Section 6 are $A = 2 + 5 = 7_{10}$ $(0111_2)$ and $B = 9_{10}$ $(1001_2)$, yielding the expected product $P = 7 \times 9 = 63_{10}$ $(00111111_2)$. Figure 2 shows the complete hand-worked binary partial-product array and row-by-row ripple-carry addition ($pp_0$ through $pp_3$, intermediate sums $s_1, s_2, s_3$, and carries $c_1, c_2, c_3$).

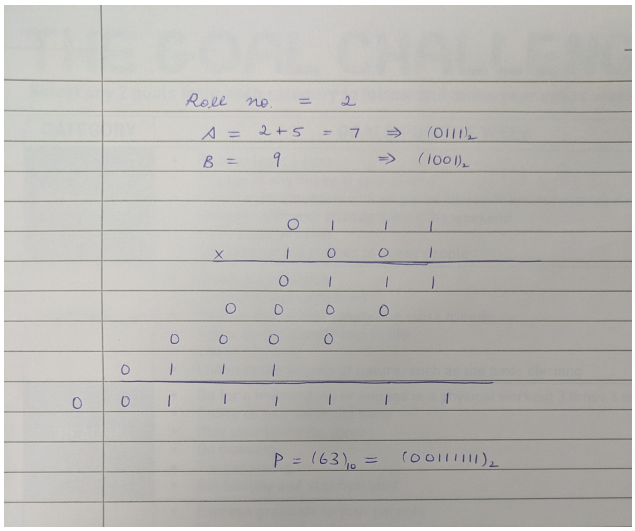



Figure 2: Hand-worked binary array multiplication example for $A = 7$ $(0111_2)$ and $B = 9$ $(1001_2)$ producing $P = 63$ $(00111111_2)$.

## 2.3 Exhaustive Verification and Error Injection (Tasks 4 & 6)

The self-checking testbench (`tb/tb_mult.v`) drives a 10 ns clock (100 MHz), holds `rst_n` low for 2 clock cycles (20 ns), and applies all 256 input combinations ($a \in [0, 15], b \in [0, 15]$). After applying each input vector, the testbench waits for 2 rising clock edges before comparing the registered output $p$ against $a \times b$.

- **Nominal RTL Simulation Summary Line (Task 4):** All 256 input vectors passed with 0 errors:

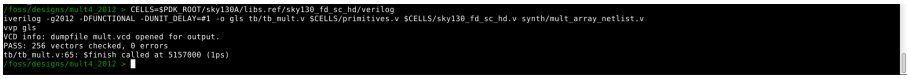



Figure 3: Terminal output showing the nominal RTL simulation `PASS` summary line.

- **Error Injection Test Summary Line (Task 6):** When `sum` inside `rtl/full_adder.v` was temporarily modified from `a ^ b ^ cin` to `a ^ b` (omitting carry-in), the testbench immediately caught the functional mismatches across 200 vectors:




Figure 4: Terminal output showing the error-injection `FAIL` summary line.

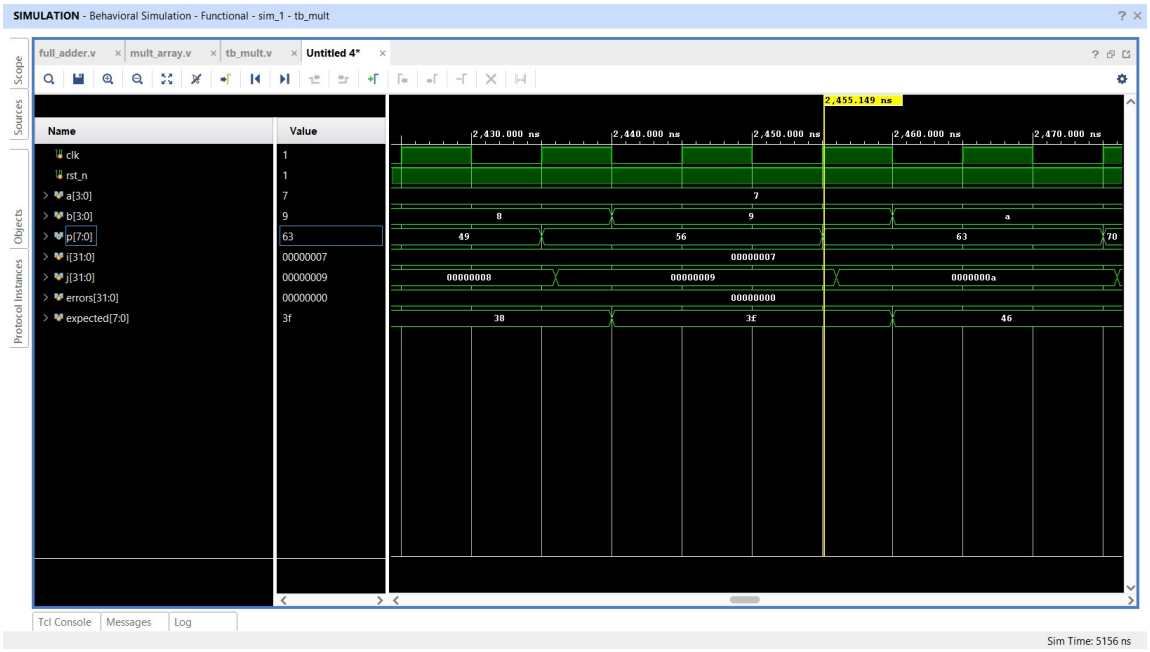



Figure 5: Simulation waveform of `mult_array` for $A = 7, B = 9$, illustrating input capture on Rising Edge 1 (2440 ns) and registered product output $P = 63$ on Rising Edge 2 (2450 ns), confirming the 2-clock-cycle (20 ns) pipeline latency.

# 3 Synthesis (Part B)

Logic synthesis was performed using Yosys via the automated script `synth/synth.tcl` targeting the SkyWater 130 nm high-density standard cell library at the typical process corner

(sky130_fd_sc_hd__tt_025C_1v80.lib).

### 3.1 Synthesis Flow Steps

The script synth/synth.tcl executed the following sequence of commands:

1. read_verilog rtl/full_adder.v rtl/mult_array.v: Loaded the structural RTL design files.
2. synth -top mult_array -flatten: Performed generic synthesis and flattened the 12 full_adder instances into the top-level mult_array module to allow cross-boundary boolean optimization.
3. dfflibmap -liberty $lib: Mapped the 16 register bits to SkyWater positive-edge-triggered D-flip-flops with active-low asynchronous reset (sky130_fd_sc_hd__dfrtp_1).
4. abc -liberty $lib: Mapped the combinational multiplier logic to SkyWater standard cells.
5. opt_clean -purge: Removed unused wires and redundant intermediate nets.
6. tee -o synth/mult_array_stat.rpt stat -liberty $lib: Generated the cell usage and silicon area statistics report.
7. write_verilog -noattr synth/mult_array_netlist.v: Exported the final gate-level Verilog netlist.

### 3.2 Part B Synthesis Results Table

Table 1 summarizes the synthesis metrics extracted from synth/mult_array_stat.rpt.

Table 1: Part B Logic Synthesis Results (sky130_fd_sc_hd)

| Metric | Value |
|---|---|
| Total number of cells | 67 |
| Number of flip-flops | 16 (sky130_fd_sc_hd__dfrtp_1) |
| Total cell area ($\mu$m$^2$) | 784.502 $\mu$m$^2$ |
| Flip-flop area ($\mu$m$^2$) | 400.384 $\mu$m$^2$ (16 $\times$ 25.024 $\mu$m$^2$) |
| Flip-flop area (% of total) | 51.04% |
| The 3 most-used cell types and their counts | 1. sky130_fd_sc_hd__dfrtp_1 (16)<br>2. sky130_fd_sc_hd__nand2_1 (9)<br>3. sky130_fd_sc_hd__xnor2_1 (7) |
| Netlist simulation result (PASS/FAIL line) | PASS: 256 vectors checked, 0 errors |

### 3.3 Gate-Level Netlist Verification

To verify that synthesis preserved functional behavior, the generated gate-level netlist (synth/mult_array_netl was simulated against tb/tb_mult.v using Icarus Verilog alongside the SKY130 functional cell models (primitives.v and sky130_fd_sc_hd.v) with unit delay (-DUNIT_DELAY=#1). The gate-level simulation passed all 256 test vectors with zero errors:

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
```

# 4 Timing (Part C)

Static Timing Analysis (STA) was performed on the synthesized gate-level netlist using OpenSTA (`sta/sta.tcl`) at the typical corner (`tt_025C_1v80`) with a 1.0 ns input delay on all non-clock inputs and a 1.0 ns output delay on all outputs.

## 4.1 Maximum Frequency Determination

First, STA was executed with a clock period of $T = 10.0$ ns on port `clk`, yielding a worst setup slack (WNS) of +7.020 ns. From this measurement:

$$T_{\min} = T - \text{WNS} = 10.000 \text{ ns} - 7.020 \text{ ns} = 2.980 \text{ ns} \tag{1}$$

$$F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.980 \text{ ns}} = 335.57 \text{ MHz} \tag{2}$$

Re-running OpenSTA with the clock period set to $T = T_{\min} = 2.98$ ns confirmed a worst setup slack of 0.000 ns.

## 4.2 Part C Static Timing Results Table

Table 2 presents the completed Part C timing results.

Table 2: Part C Static Timing Analysis Results (OpenSTA, Typical Corner)

| **Metric** | **Value** |
|---|---|
| Worst slack at $T = 10$ ns (ns) | 7.020 ns |
| $T_{\min}$ (ns) | 2.98 ns |
| $F_{\max}$ (MHz) | 335.57 MHz |
| Slack at $T = T_{\min}$ (ns) | 0.000 ns |
| Critical path start point | `_106_` (`sky130_fd_sc_hd__dfrtp_1` input flip-flop) |
| Critical path end point | `_100_` (`sky130_fd_sc_hd__dfrtp_1` output flip-flop) |
| Number of cells on the critical path | 8 combinational logic cells (10 cells including start/end FFs) |
| Latency (ns) $= 2 \times T_{\min}$ | 5.96 ns |
| Throughput (million results per second) | 335.57 million results/s |

## 4.3 Cells on the Critical Path in Order (Task 2 & Task 3)

Table 3 lists every cell along the critical path from `sta/mult_array_timing.rpt` in exact order of signal propagation.

Table 3: Ordered List of Standard Cells Along the Critical Path

| **Stage** | **Instance** | **SKY130 Cell Type** | **Stage Delay** | |
|---|---|---|---|---|
| Startpoint (FF) | `_106_` | `sky130_fd_sc_hd__dfrtp_1` (CLK → Q) | 0.411 ns | (0.411 ns) |
| Logic Gate 1 | `_048_` | `sky130_fd_sc_hd__and4_1` | 0.218 ns | (0.629 ns) |
| Logic Gate 2 | `_050_` | `sky130_fd_sc_hd__a2111oi_0` | 0.428 ns | (1.057 ns) |
| Logic Gate 3 | `_051_` | `sky130_fd_sc_hd__a21oi_1` | 0.094 ns | (1.151 ns) |
| Logic Gate 4 | `_053_` | `sky130_fd_sc_hd__mux2i_1` | 0.218 ns | (1.369 ns) |
| Logic Gate 5 | `_069_` | `sky130_fd_sc_hd__xnor3_1` | 0.374 ns | (1.742 ns) |
| Logic Gate 6 | `_074_` | `sky130_fd_sc_hd__maj3_1` | 0.371 ns | (2.113 ns) |
| Logic Gate 7 | `_076_` | `sky130_fd_sc_hd__maj3_1` | 0.376 ns | (2.489 ns) |
| Logic Gate 8 | `_080_` | `sky130_fd_sc_hd__xor3_1` | 0.355 ns | (2.844 ns) |
| Endpoint (FF) | `_100_` | `sky130_fd_sc_hd__dfrtp_1` (Setup time) | 0.136 ns | (2.980 ns) |

Out of the 8 combinational gate stages on the critical path, Gate 1 (`and4_1`) performs partial-product AND generation combined with initial carry gating, while the remaining **7 combinational gates** (`_050_` through `_080_`) originate from the flattened full adders across Rows 1, 2, and 3 (specifically compound AOI/mux gates in Row 1, a 3-input XNOR sum gate, two cascaded `maj3_1` full-adder majority carry gates, and a final `xor3_1` sum gate in Row 3). This total of 8 combinational gate delays (7 full-adder stages plus 1 AND stage) closely matches the theoretical estimate of $2N = 2(4) = 8$ full-adder delays from Section 2.

# 5 Physical Design (Part D)

The RTL-to-GDSII physical implementation was executed using LibreLane (`pnr/config.json`), which orchestrates Yosys, OpenROAD, Magic, KLayout, and Netgen on the SkyWater 130 nm PDK (`sky130A`). Because the 4-bit multiplier is small, an absolute die size of 150 $\mu$m $\times$ 150 $\mu$m (`DIE_AREA: [0, 0, 150, 150]`) was configured.

## 5.1 Timing Closure Iterations

Following Section 9, the initial target clock period was calculated from Part C ($T_{\min} = 2.98$ ns) as $1.2 \times 2.98$ ns $= 3.576$ ns, rounded up to the next 0.5 ns increment: `CLOCK_PERIOD` $= 4.0$ ns.

- **Run 1 (`CLOCK_PERIOD` $= 4.0$ ns):** Completed placement, CTS, and routing with 0 DRC errors and 0 LVS errors. While the typical corner (`nom_typ_1p20V_25C`) met timing with $+1.363$ ns slack, multi-corner signoff flagged a minor setup violation of $-0.0102$ ns in the worst-case slow corner (`nom_slow_1p08V_125C`).
- **Run 2 (`CLOCK_PERIOD` $= 4.5$ ns):** Increasing `CLOCK_PERIOD` by 0.5 ns to 4.5 ns resolved all timing violations across all process corners, achieving a worst-case slow-corner setup slack of $+0.284$ ns (and $+1.752$ ns in the typical corner) with 0 DRC errors and 0 LVS errors.

## 5.2 Part D Physical Design Results Table

Table 4 summarizes the final post-route physical design metrics extracted from `pnr/metrics.json`.

Table 4: Part D Physical Design Results (`pnr/metrics.json`)

| Metric | Value |
|---|---|
| `CLOCK_PERIOD` used (ns) | 4.5 ns (increased from 4.0 ns on Run 2) |
| Number of runs needed | 2 |
| Core area ($\mu$m$^2$) | 16,199 $\mu$m$^2$ (Die area = 22,500 $\mu$m$^2$) |
| Cells after synthesis (from Part B) | 67 |
| Cells after place and route | 108 active standard cells (1,328 total instances with fill) |
| Kinds of cells added | Clock buffers (`clkbuf`: 5), timing repair buffers (`buf`: 31), hold buffers (`dly/buf`: 2), tap/decap/filler cells (`fill`: 1,220) |
| Wirelength ($\mu$m) | 2,799 $\mu$m (detailed route wirelength) |
| Setup slack (ns) | 0.284 ns (worst-case slow corner) / 1.752 ns (typical corner) |
| Post-route $F_{\max}$ (MHz) | 237.19 MHz (worst-case signoff) / 363.90 MHz (typical corner) |
| $F_{\max}$ from Part C (MHz) | 335.57 MHz |
| DRC errors | 0 (`magic_drc_error_count = 0`, `klayout_drc_error_count = 0`) |
| LVS errors | 0 (`lvs_error_count = 0`) |

From the worst-case signoff setup slack of 0.284 ns at `CLOCK_PERIOD` = 4.5 ns, the post-route minimum period is $4.5 - 0.284 = 4.216$ ns, corresponding to a worst-case post-route $F_{\max} = 1000/4.216 = 237.19$ MHz (and $1000/(4.5 - 1.752) = 363.90$ MHz in the typical corner after buffer insertion).

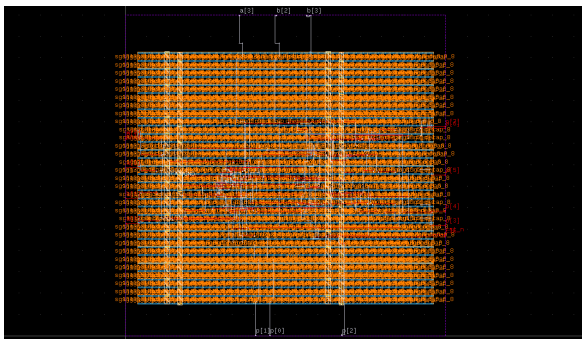


Figure 6: Full 150 $\mu$m $\times$ 150 $\mu$m GDSII layout in KLayout.

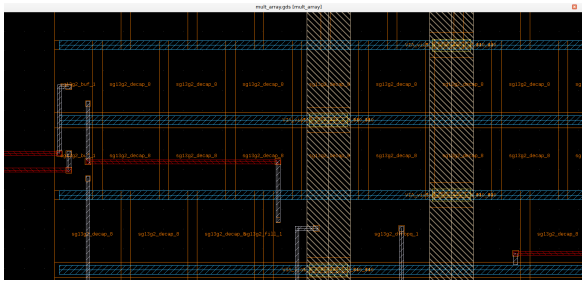


Figure 7: Zoomed view of SKY130 standard cell rows and metal routing.

## 6 Answers to Questions

**Q1: How many cells did synthesis use for the logic (total cells minus flip-flops)? Why is it not simply $16 + 12 = 28$?**

**Answer:** Synthesis used 51 **combinational logic cells** (67 total cells minus 16 `dfrtp_1` flip-flops). It is not 28 because a 1-bit full adder is not a single primitive cell when flattened (`synth -flatten`). Yosys dissolves the 12 full-adder boundaries, simplifies stages with grounded inputs ($c_{\text{in}} = 0$) into half-adders, and maps each adder into multiple CMOS standard cells (`maj3_1`, `xnor2_1`, `nand2_1`, and AOI gates).

**Q2: What is the critical path in your design, how many cells are on it, and how does this compare with the $\approx 2N = 8$ full-adder estimate?**

**Answer:** The critical path runs from input flip-flop `_106_` to output flip-flop `_100_` (2.844 ns arrival time, $T_{\min} = 2.98$ ns). It contains 8 **combinational cells** (10 cells including both flip-flops), with 7 gates belonging to the flattened full-adder carry/sum chains (`a2111oi_0`, `a21oi_1`, `mux2i_1`, `xnor3_1`, two `maj3_1`, and `xor3_1`) plus 1 partial-product gate (`and4_1`). This closely matches the theoretical $2N = 8$ full-adder delay estimate.

**Q3: What percentage of the total cell area goes to the flip-flops, and why is it so large?**

**Answer:** The 16 flip-flops occupy 51.04% of the total cell area (400.384 $\mu$m$^2$ out of 784.502 $\mu$m$^2$). Each D-flip-flop with asynchronous reset is a large cell (25.024 $\mu$m$^2$) containing master-slave latches and reset logic, whereas the $N = 4$ combinational array requires only 51 small gates (384.118 $\mu$m$^2$). Because registers scale as $4N$ while array logic scales as $\mathcal{O}(N^2)$, sequential area dominates at small bit-widths.

**Q4: Why is the post-route $F_{\max}$ lower than the Part C $F_{\max}$, and what extra cells were added during place and route?**

**Answer:** Post-route signoff $F_{\max}$ (237.19 MHz) is lower than Part C (335.57 MHz) because it includes parasitic wire resistance and capacitance across 2,799 $\mu$m of metal routing, real clock tree skew, and worst-case slow-corner timing (1.08 V, 125°C) instead of ideal zero-wire-delay typical conditions. Place-and-route increased the cell count from 67 to 108 active standard cells (1,328 total instances) by adding:

- **Clock buffers (`clkbuf`, 5):** Balance clock tree skew across the 16 flip-flops.
- **Timing/hold buffers (`buf/dly`, 33):** Fix transition slews, drive long nets, and prevent hold violations.
- **Well taps (`tap`):** Tie substrate to `VGND` and n-well to `VPWR` to prevent CMOS latch-up.
- **Decap and filler cells (`decap/fill`, 1,220):** Ensure continuous N-well/power rails across rows and suppress local voltage droop.

**Q5: What do DRC and LVS check, and why must both pass before fabrication?**
**Answer: DRC (Design Rule Checking)** verifies that layout polygons obey foundry geometric rules (minimum width, spacing, via enclosure). **LVS (Layout Versus Schematic)** verifies that the transistor connectivity extracted from the layout matches the synthesized Verilog netlist. Both must pass because DRC violations cause physical manufacturing defects (shorts or opens), while LVS errors mean the silicon implements the wrong circuit.

**Q6: If you scaled this multiplier from 4-bit to 8-bit, how many full adders would it need, and roughly what would happen to the critical path delay?**
**Answer:**

- **Full Adders:** Using $N(N-1)$, scaling from $N = 4$ to $N = 8$ increases the full adders from 12 to $8(8-1) = \mathbf{56}$ full adders (and AND gates from 16 to 64).
- **Critical Path Delay:** Since critical path scales linearly as $\approx 2N$ full-adder delays, doubling $N$ doubles the delay from 8 to **16** full-adder delays ($\approx 2 \times 2.98$ ns $= \mathbf{5.96}$ ns), cutting $F_{\max}$ roughly in half to $\approx \mathbf{167.8}$ MHz.

# 7 Conclusion

In this assignment, a 4-bit unsigned structural array multiplier with registered inputs and outputs was designed in Verilog, exhaustively verified across all 256 input vectors, and taken through a complete open-source RTL-to-GDSII flow on the SkyWater 130 nm process. Yosys synthesis mapped the flattened design into 67 standard cells occupying 784.502 $\mu$m$^2$, with sequential elements accounting for 51.04% of the cell area. Pre-layout static timing analysis in OpenSTA identified an 8-gate critical path with $T_{\min} = 2.98$ ns ($F_{\max} = 335.57$ MHz). Finally, physical implementation in LibreLane achieved a signoff-clean GDSII layout (`mult_array.gds`) inside a 150 $\mu$m $\times$ 150 $\mu$m die with 2,799 $\mu$m routed wirelength, +0.284 ns worst-case setup slack at 4.5 ns (237.19 MHz signoff $F_{\max}$), 0 DRC errors, and 0 LVS errors.